

✓ 1 - IMPORT LIBRARIES AND DATASETS

```
# Import required libraries
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
import matplotlib.cm as cm
from imblearn.over_sampling import SMOTE
from collections import Counter
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
import xgboost as xgb
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_curve
from sklearn.metrics import auc
```

```
# Import dataset
```

```
data = pd.read_csv("/content/drive/MyDrive/Data Set/Airline_Delays_Reported.csv")
data.head()
```

```
data_ariline = pd.read_csv("/content/drive/MyDrive/Data Set/airlines.csv")
data_ariline.head()
```

	IATA_CODE	AIRLINE_NAME	
0	UA	United Air Lines Inc.	
1	AA	American Airlines Inc.	
2	US	US Airways Inc.	
3	F9	Frontier Airlines Inc.	
4	B6	JetBlue Airways	

Next steps:

[Generate code with data_ariline](#)[View recommended plots](#)[New interactive sheet](#)

✓ 2 - DATA PRE-PROCESSING

```
# Eliminate duplicates
```

```
data.duplicated().sum()
data.shape
```

```
(504446, 27)
```

```
# Handling NULL values
```

```
data.isna().sum()
```



	0
DAY	0
DAY_OF_WEEK	0
AIRLINE	0
FLIGHT_NUMBER	0
TAIL_NUMBER	0
ORIGIN_AIRPORT	0
DESTINATION_AIRPORT	0
SCHEDULED_DEPARTURE	0
DEPARTURE_TIME	0
DEPARTURE_DELAY	0
TAXI_OUT	0
WHEELS_OFF	0
SCHEDULED_TIME	0
ELAPSED_TIME	0
AIR_TIME	0
DISTANCE	0
WHEELS_ON	0
TAXI_IN	0
SCHEDULED_ARRIVAL	0
ARRIVAL_TIME	0
ARRIVAL_DELAY	0
AIR_SYSTEM_DELAY	0
CHK_IMM_DELAY	0
SECURITY_DELAY	0
AIRLINE_DELAY	0
LATE_AIRCRAFT_DELAY	0
WEATHER_DELAY	0

dtype: int64

```
# Create AIRLINE_NAME and VLOOKUP from data_ariline dataset by AIRLINE column

data_one = data.merge(data_ariline, left_on='AIRLINE', right_on='IATA_CODE', how='left')
data_one.head()
```



	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT	DESTINATION_AIRPORT	SCHEDULED_DEPARTURE
0	1	4	NK	520	N525NK	LAS	MCI	55
1	1	4	NK	597	N528NK	MSP	FLL	115
2	1	4	AA	2392	N3HRAA	DEN	MIA	120
3	1	4	NK	168	N629NK	PHX	ORD	125
4	1	4	B6	2134	N307JB	SJU	MCO	400

5 rows x 29 columns

```
# Eliminate unwanted columns

columns_drop = [
    'TAIL_NUMBER', 'DEPARTURE_TIME', 'WHEELS_OFF', 'SCHEDULED_TIME',
    'ELAPSED_TIME', 'AIR_TIME', 'DISTANCE', 'WHEELS_ON',
    'ARRIVAL_TIME', 'AIRLINE'
]
```

```
data_two = data_one.drop(columns=columns_drop)
data_two.head()
```



	DAY	DAY_OF_WEEK	FLIGHT_NUMBER	ORIGIN_AIRPORT	DESTINATION_AIRPORT	SCHEDULED_DEPARTURE	DEPARTURE_DELAY	TAXI
0	1	4	520	LAS	MCI	55	25	
1	1	4	597	MSP	FLL	115	12	
2	1	4	2392	DEN	MIA	120	21	
3	1	4	168	PHX	ORD	125	72	
4	1	4	2134	SJU	MCO	400	95	

```
# Define new column order

new_column_order = [
    'DAY', 'DAY_OF_WEEK', 'FLIGHT_NUMBER', 'IATA_CODE', 'AIRLINE_NAME', 'ORIGIN_AIRPORT', 'DESTINATION_AIRPORT', 'SCHEDULED_DEPARTURE', 'DEPARTURE_DELAY', 'ARRIVAL_DELAY', 'SECURITY_DELAY', 'WEATHER_DELAY', 'LATE_AIRCRAFT_DELAY', 'AIR_SYSTEM_DELAY', 'CHK_IMM_DELAY'
]
data_three = data_two[new_column_order]
data_three.head()
```



	DAY	DAY_OF_WEEK	FLIGHT_NUMBER	IATA_CODE	AIRLINE_NAME	ORIGIN_AIRPORT	DESTINATION_AIRPORT	SCHEDULED_DEPARTURE	DEPARTURE_DELAY	ARRIVAL_DELAY	SECURITY_DELAY	WEATHER_DELAY	LATE_AIRCRAFT_DELAY	AIR_SYSTEM_DELAY	CHK_IMM_DELAY
0	1	4	520	NK	Spirit Air Lines	LAS	MCI	55	25	15	10	5	10	5	5
1	1	4	597	NK	Spirit Air Lines	MSP	FLL	115	12	20	15	10	15	10	10
2	1	4	2392	AA	American Airlines Inc.	DEN	MIA	120	21	25	20	15	20	15	15
3	1	4	168	NK	Spirit Air Lines	PHX	ORD	125	72	30	25	20	25	20	20
4	1	4	2134	B6	JetBlue Airways	SJU	MCO	400	95	40	35	30	35	30	30

```
# Eliminate outliers DEPARTURE_DELAY

Q1 = data_three["DEPARTURE_DELAY"].quantile(0.25)
Q3 = data_three["DEPARTURE_DELAY"].quantile(0.75)
IQR = Q3 - Q1
LB = Q1 - 1.5 * IQR
UB = Q3 + 1.5 * IQR

data_four = data_three[(data_three["DEPARTURE_DELAY"] >= LB) & (data_three["DEPARTURE_DELAY"] <= UB)]
data_four.shape

Q1 = data_four["DEPARTURE_DELAY"].quantile(0.25)
Q3 = data_four["DEPARTURE_DELAY"].quantile(0.75)
IQR = Q3 - Q1
LB = Q1 - 1.5 * IQR
UB = Q3 + 1.5 * IQR

data_five = data_four[(data_four["DEPARTURE_DELAY"] >= LB) & (data_four["DEPARTURE_DELAY"] <= UB)]
data_five.shape

Q1 = data_five["DEPARTURE_DELAY"].quantile(0.25)
Q3 = data_five["DEPARTURE_DELAY"].quantile(0.75)
IQR = Q3 - Q1
LB = Q1 - 1.5 * IQR
UB = Q3 + 1.5 * IQR

data_six = data_five[(data_five["DEPARTURE_DELAY"] >= LB) & (data_five["DEPARTURE_DELAY"] <= UB)]
data_six.shape

Q1 = data_six["DEPARTURE_DELAY"].quantile(0.25)
Q3 = data_six["DEPARTURE_DELAY"].quantile(0.75)
IQR = Q3 - Q1
LB = Q1 - 1.5 * IQR
UB = Q3 + 1.5 * IQR

data_seven = data_six[(data_six["DEPARTURE_DELAY"] >= LB) & (data_six["DEPARTURE_DELAY"] <= UB)]
data_seven.shape
```

```
Q1 = data_seven["DEPARTURE_DELAY"].quantile(0.25)
Q3 = data_seven["DEPARTURE_DELAY"].quantile(0.75)
IQR = Q3 - Q1
LB = Q1 - 1.5 * IQR
UB = Q3 + 1.5 * IQR

data_eight = data_seven[(data_seven["DEPARTURE_DELAY"] >= LB) & (data_seven["DEPARTURE_DELAY"] <= UB)]
data_eight.shape

Q1 = data_eight["DEPARTURE_DELAY"].quantile(0.25)
Q3 = data_eight["DEPARTURE_DELAY"].quantile(0.75)
IQR = Q3 - Q1
LB = Q1 - 1.5 * IQR
UB = Q3 + 1.5 * IQR

data_nine = data_eight[(data_eight["DEPARTURE_DELAY"] >= LB) & (data_eight["DEPARTURE_DELAY"] <= UB)]
data_nine.shape

↗ (404694, 17)
```

✓ 3 - DATA ANALYSIS

```
# Print the statistical summary

summary_stats = data[['DEPARTURE_DELAY', 'ARRIVAL_DELAY', 'SCHEDULED_DEPARTURE', 'SCHEDULED_ARRIVAL']].describe()
summary_stats
```

↗

	DEPARTURE_DELAY	ARRIVAL_DELAY	SCHEDULED_DEPARTURE	SCHEDULED_ARRIVAL	
count	504446.000000	504446.000000	504446.000000	504446.000000	
mean	26.650046	26.847492	1384.294983	1567.999385	
std	50.327668	51.897935	453.987203	478.834698	
min	-42.000000	-78.000000	1.000000	1.000000	
25%	2.000000	2.000000	1017.000000	1220.000000	
50%	10.000000	11.000000	1410.000000	1620.000000	
75%	32.000000	33.000000	1744.000000	1947.000000	
max	1988.000000	1971.000000	2359.000000	2359.000000	

Next steps:

[Generate code with summary_stats](#)

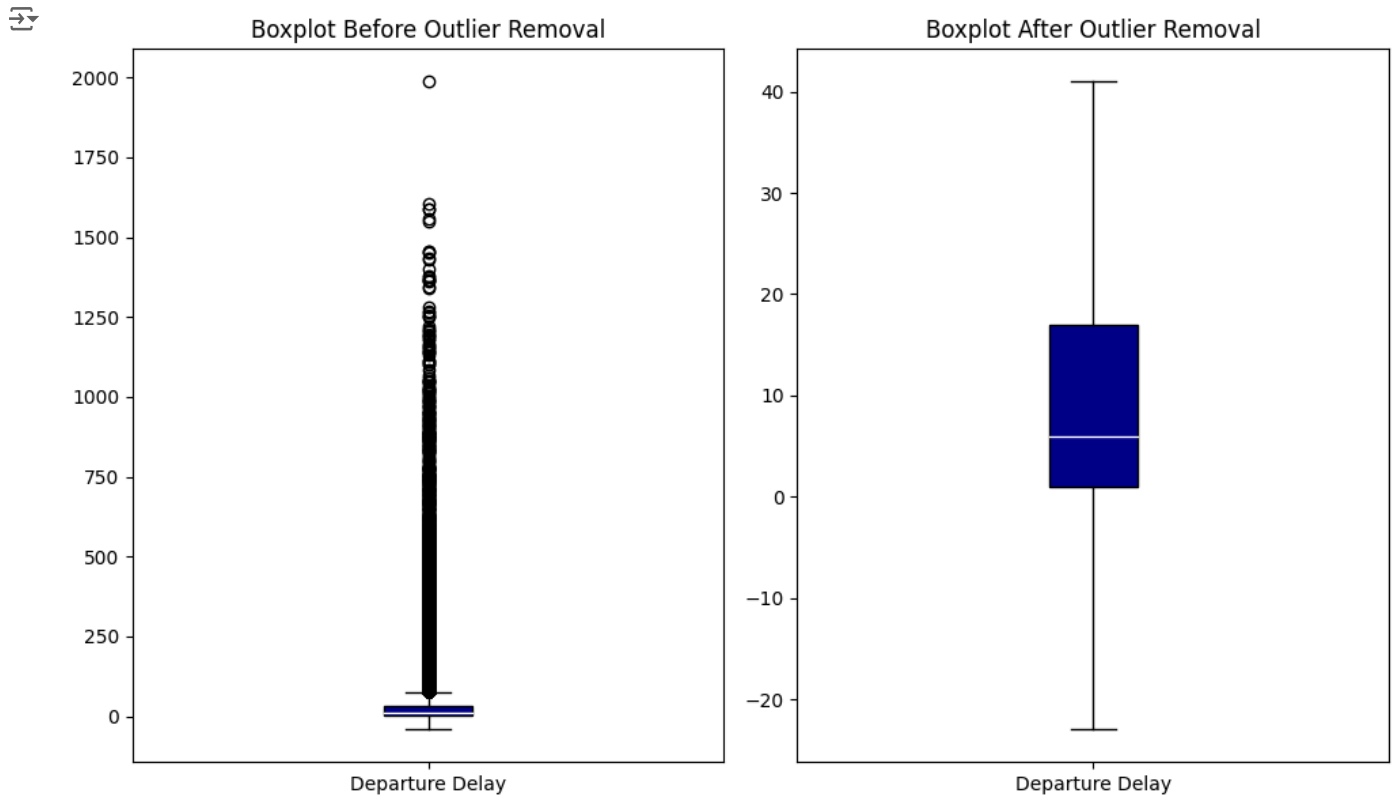
[View recommended plots](#)

[New interactive sheet](#)

```
# Boxplot for outlier state comparison

plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.boxplot([data_three['DEPARTURE_DELAY']], patch_artist=True,
            boxprops=dict(facecolor='darkblue'),
            medianprops=dict(color='white'))
plt.title('Boxplot Before Outlier Removal')
plt.xticks([1], ['Departure Delay'])

plt.subplot(1, 2, 2)
plt.boxplot([data_nine['DEPARTURE_DELAY']], patch_artist=True,
            boxprops=dict(facecolor='darkblue'),
            medianprops=dict(color='white'))
plt.title('Boxplot After Outlier Removal')
plt.xticks([1], ['Departure Delay'])
plt.tight_layout()
plt.show()
```

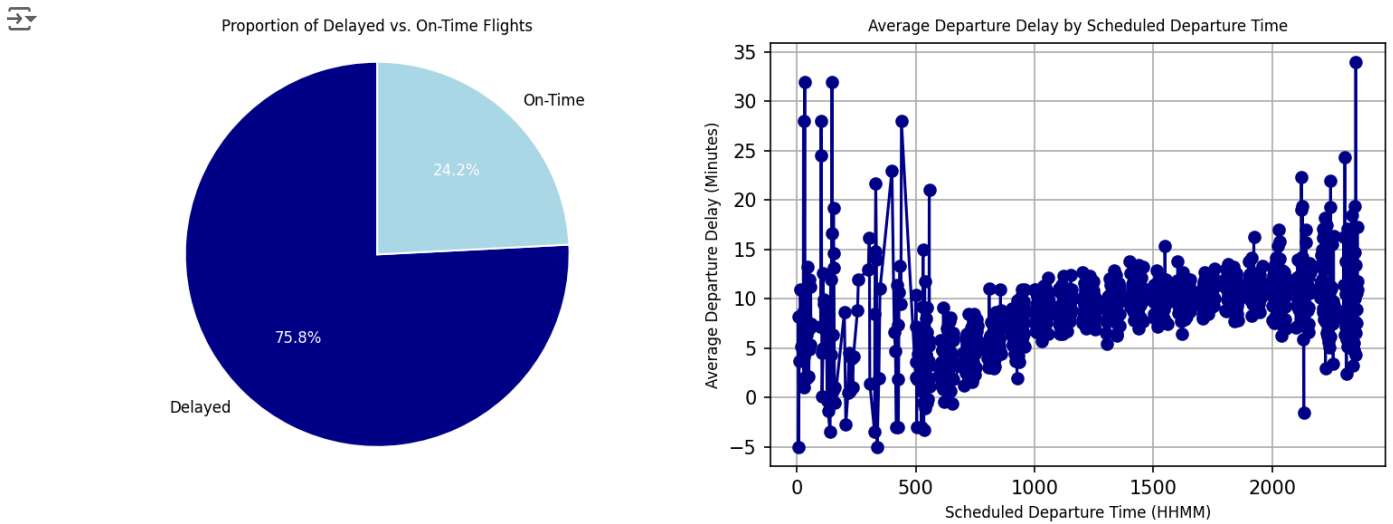


```
# Delayed vs ontime analysis
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4), dpi=150)
data_nine['Delayed'] = (data_nine['DEPARTURE_DELAY'] > 0)
flight_counts = data_nine['Delayed'].value_counts()
labels = ['Delayed', 'On-Time']
colors = ['#00008b', '#add8e6'] # Dark blue and light blue
ax1.pie(flight_counts, labels=labels, autopct='%1.1f%%', startangle=90,
        colors=colors, textprops={'fontsize': 8, 'color': 'white'},
        wedgeprops={'edgecolor': 'white'})
for text in ax1.texts:
    if text.get_text() in labels:
        text.set_color('black')
ax1.axis('equal')
ax1.set_title('Proportion of Delayed vs. On-Time Flights', fontsize=8)
```

```
# Depature delay by depature time
```

```
departure_vs_delay = data_nine.groupby('SCHEDULED_DEPARTURE')['DEPARTURE_DELAY'].mean()
ax2.plot(departure_vs_delay.index, departure_vs_delay.values, marker='o', linestyle='-', color='darkblue')
ax2.set_title('Average Departure Delay by Scheduled Departure Time', fontsize=8)
ax2.set_xlabel('Scheduled Departure Time (HHMM)', fontsize=8)
ax2.set_ylabel('Average Departure Delay (Minutes)', fontsize=8)
ax2.grid(True)
plt.tight_layout()
plt.show()
```

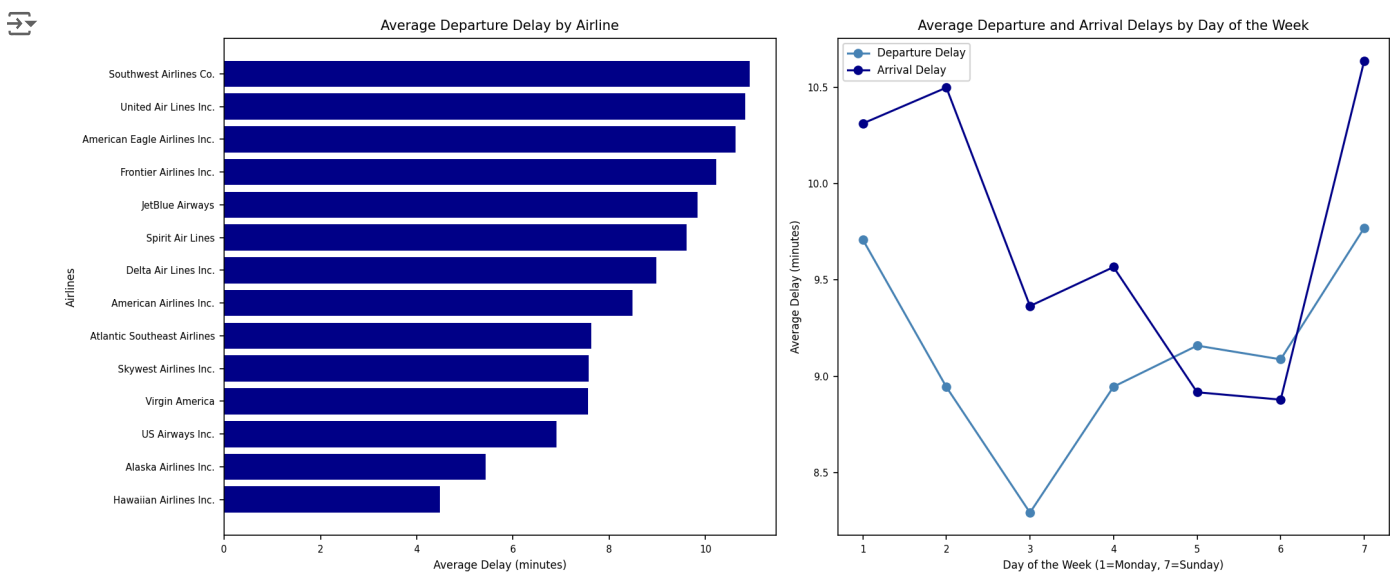


```
# Average Departure Delay by Airline
```

```
fig, axs = plt.subplots(1, 2, figsize=(14, 6), dpi=150)
airline_delay = data_nine.groupby('AIRLINE_NAME')['DEPARTURE_DELAY'].mean().sort_values()
axs[0].barh(airline_delay.index, airline_delay.values, color='darkblue')
axs[0].set_title('Average Departure Delay by Airline', fontsize=10)
axs[0].set_xlabel('Average Delay (minutes)', fontsize=8)
axs[0].set_ylabel('Airlines', fontsize=8)
axs[0].tick_params(axis='x', labels=7)
axs[0].tick_params(axis='y', labels=7)
```

```
# Line plot of average departure and arrival delays by the week
```

```
day_of_week_delays = data_nine.groupby('DAY_OF_WEEK').agg({
    'DEPARTURE_DELAY': 'mean',
    'ARRIVAL_DELAY': 'mean'
})
axs[1].plot(day_of_week_delays.index, day_of_week_delays['DEPARTURE_DELAY'], label='Departure Delay', marker='o', color='darkblue')
axs[1].plot(day_of_week_delays.index, day_of_week_delays['ARRIVAL_DELAY'], label='Arrival Delay', marker='o', color='darkblue')
axs[1].set_title('Average Departure and Arrival Delays by Day of the Week', fontsize=10)
axs[1].set_xlabel('Day of the Week (1=Monday, 7=Sunday)', fontsize=8)
axs[1].set_ylabel('Average Delay (minutes)', fontsize=8)
axs[1].tick_params(axis='x', labels=7)
axs[1].tick_params(axis='y', labels=7)
axs[1].legend(fontsize=8)
plt.tight_layout()
plt.show()
```

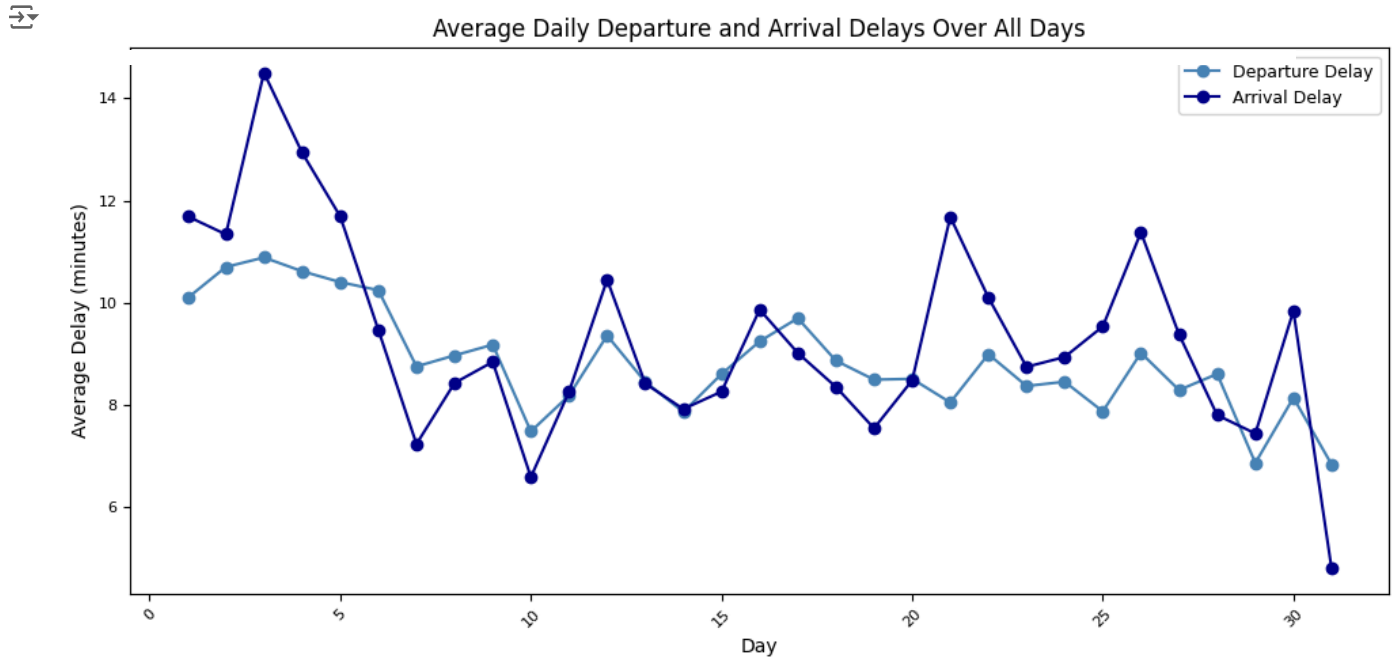


```
# Time series chart for average daily departure and arrival delays
```

```

daily_delays_all = data_nine.groupby('DAY').agg({
    'DEPARTURE_DELAY': 'mean',
    'ARRIVAL_DELAY': 'mean'
})
plt.figure(figsize=(10, 5))
plt.plot(daily_delays_all.index, daily_delays_all['DEPARTURE_DELAY'], label='Departure Delay', marker='o', color='steelblue')
plt.plot(daily_delays_all.index, daily_delays_all['ARRIVAL_DELAY'], label='Arrival Delay', marker='o', color='darkblue')
plt.title('Average Daily Departure and Arrival Delays Over All Days', fontsize=12)
plt.xlabel('Day', fontsize=10)
plt.ylabel('Average Delay (minutes)', fontsize=10)
plt.xticks(fontsize=8, rotation=45)
plt.yticks(fontsize=8)
plt.legend(fontsize=9)
plt.tight_layout()
plt.show()

```



```
# Histogram for all DEPARTURE_DELAY values
```

```

fig, ax = plt.subplots(1, 2, figsize=(12, 6), dpi=200)
ax[0].hist(data_nine['DEPARTURE_DELAY'].dropna(), bins=20, color='darkblue', edgecolor='white', alpha=0.7)
ax[0].set_title('Departure Delay (All)', fontsize=12)
ax[0].set_xlabel('Departure Delay', fontsize=10)
ax[0].set_ylabel('Frequency', fontsize=10)
ax[0].tick_params(axis='both', which='major', labelsize=10)

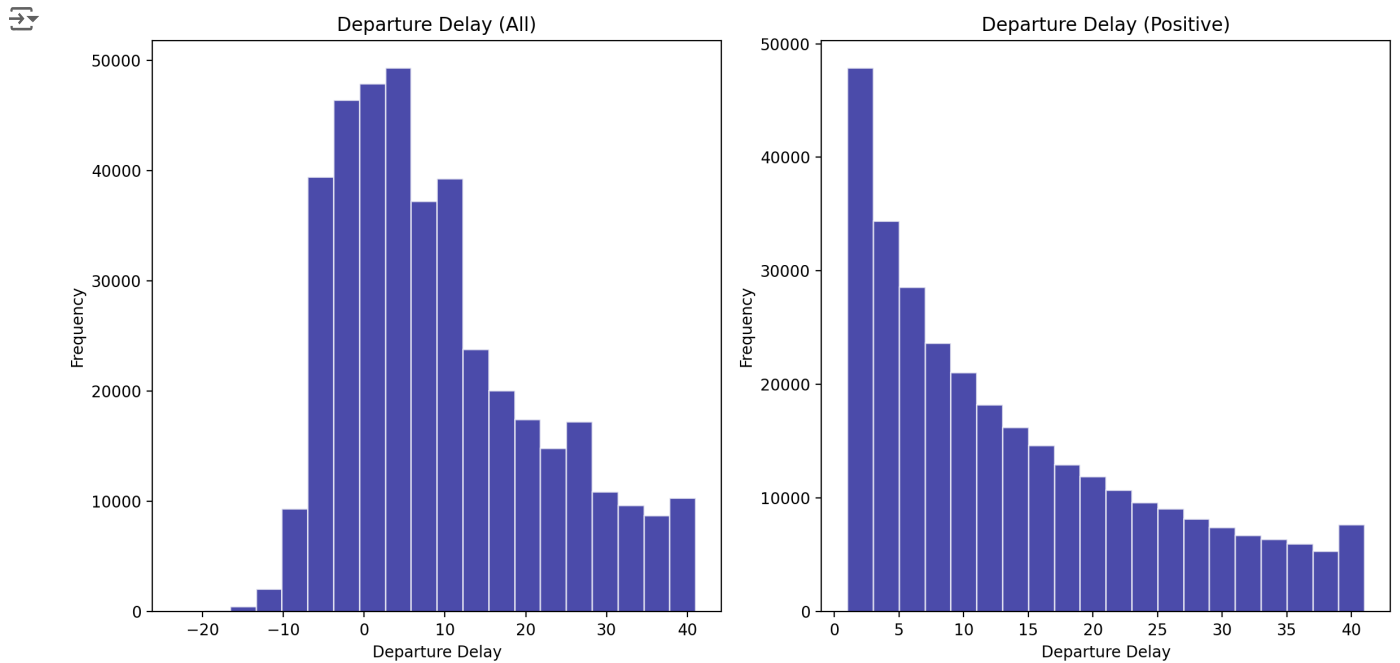
```

```
# Histogram for positive DEPARTURE_DELAY values
```

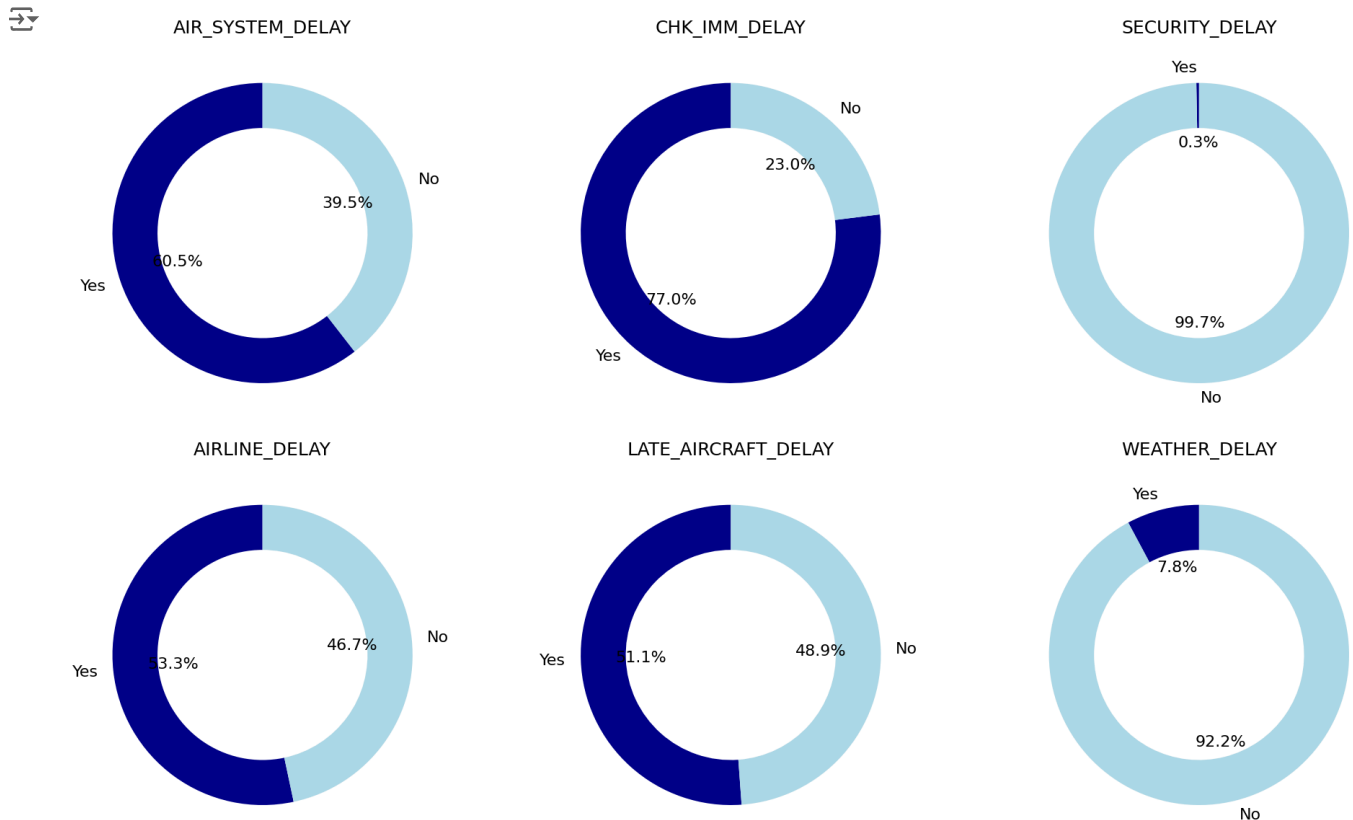
```

positive_delays = data_nine[data_nine['DEPARTURE_DELAY'] > 0]['DEPARTURE_DELAY'].dropna()
ax[1].hist(positive_delays, bins=20, color='darkblue', edgecolor='white', alpha=0.7)
ax[1].set_title('Departure Delay (Positive)', fontsize=12)
ax[1].set_xlabel('Departure Delay', fontsize=10)
ax[1].set_ylabel('Frequency', fontsize=10)
ax[1].tick_params(axis='both', which='major', labelsize=10)
plt.tight_layout()
plt.show()

```



```
# Analysis of delay reasons
delay_reasons = ['AIR_SYSTEM_DELAY', 'CHK_IMM_DELAY', 'SECURITY_DELAY',
                 'AIRLINE_DELAY', 'LATE_AIRCRAFT_DELAY', 'WEATHER_DELAY']
fig, axes = plt.subplots(2, 3, figsize=(10, 6), dpi=200)
for i, reason in enumerate(delay_reasons):
    counts = data_nine[reason].value_counts().reindex(['Yes', 'No'], fill_value=0)
    ax = axes[i // 3, i % 3]
    wedges, texts, autotexts = ax.pie(counts, labels=counts.index, autopct='%1.1f%%', startangle=90,
                                       colors=['darkblue', 'lightblue'], wedgeprops={'width': 0.3})
    for text in texts + autotexts:
        text.set_fontsize(8)
    ax.set_title(f'{reason}', fontsize=9)
plt.tight_layout()
plt.show()
```



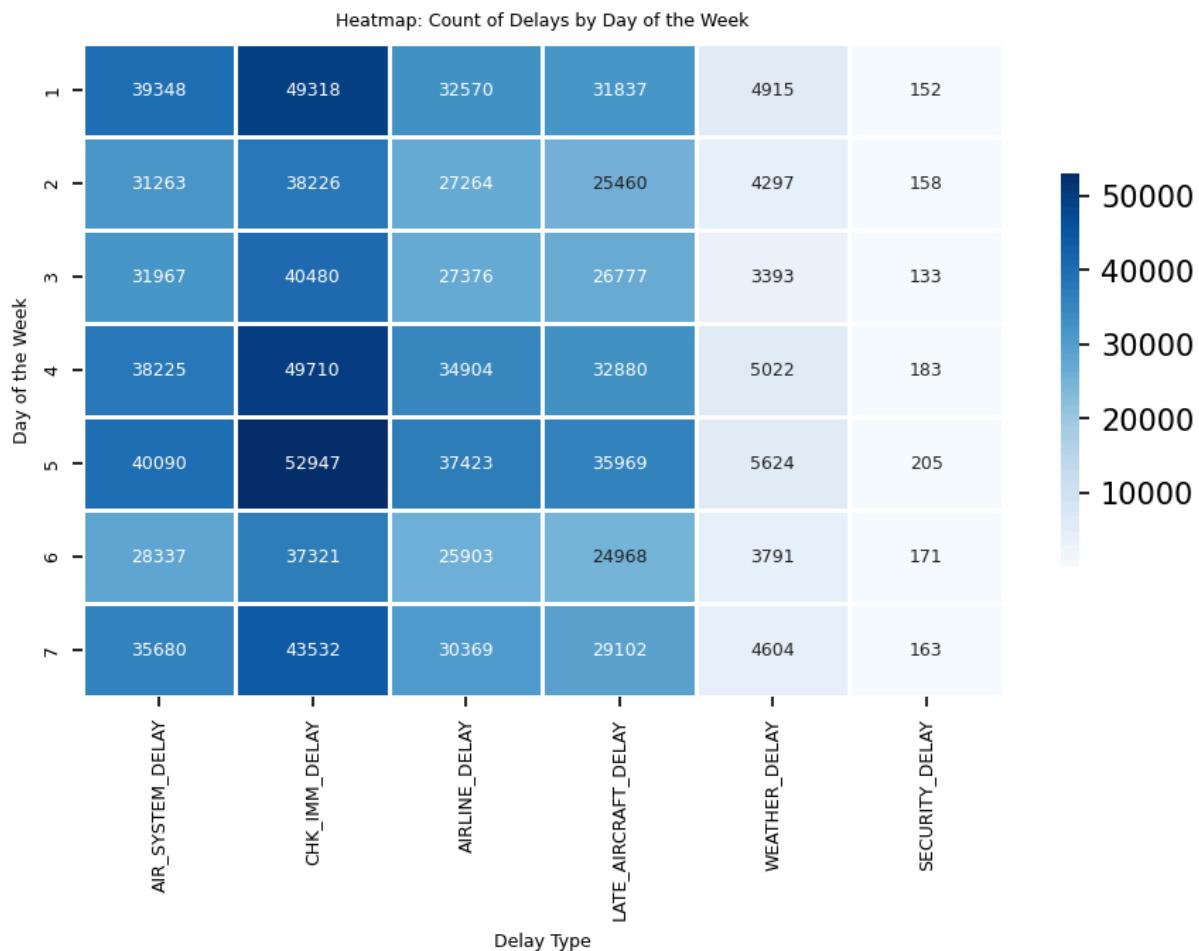

```
# Heatmap for count of delays by week
columns_to_analyze = ['AIR_SYSTEM_DELAY', 'CHK_IMM_DELAY', 'AIRLINE_DELAY',
                      'LATE_AIRCRAFT_DELAY', 'WEATHER_DELAY', 'SECURITY_DELAY']

def encode_delay(value):
    if isinstance(value, str):
        return 1 if value == 'Yes' else 0
    elif isinstance(value, (int, float)):
        return 1 if value > 0 else 0
    else:
        return 0

data_nine[columns_to_analyze] = data_nine[columns_to_analyze].applymap(encode_delay)
#print("Encoded Data Sample:")
#print(data_nine[columns_to_analyze].head())
delay_sums_by_day = data_nine.groupby('DAY_OF_WEEK')[columns_to_analyze].sum()
#print("Aggregated Delay Data by Day of Week:")
#print(delay_sums_by_day)
delay_sums_by_day = delay_sums_by_day.fillna(0)
if (delay_sums_by_day.values == 0).all():
    print("All values are zero. Heatmap cannot be displayed because no delays are recorded.")
else:
    plt.figure(figsize=(7, 4), dpi=150)
    sns.heatmap(delay_sums_by_day, annot=True, fmt='d', cmap='Blues', linewidths=0.5,
                annot_kws={"size": 6}, cbar_kws={"shrink": 0.6})
    plt.xlabel('Delay Type', fontsize=6)
    plt.ylabel('Day of the Week', fontsize=6)
    plt.title('Heatmap: Count of Delays by Day of the Week', fontsize=6)
    plt.xticks(fontsize=6)
    plt.yticks(fontsize=6)
    plt.show()
```

<ipython-input-16-ac7009e30b7d>:11: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map inst

data_nine[columns_to_analyze] = data_nine[columns_to_analyze].applymap(encode_delay)



✓ 4 - MachineLearning IMPLEMENTATION

4.1 - RandomForestClassifier - Flight Status

```
# RandomForestClassifier to predic flight status

threshold = 0
data_nine['DEPARTURE_DELAY_BINARY'] = data_nine['DEPARTURE_DELAY'].apply(lambda x: 1 if x > threshold else 0)

features_RFC = ['DAY', 'DAY_OF_WEEK', 'FLIGHT_NUMBER', 'IATA_CODE', 'SCHEDULED_DEPARTURE', 'SCHEDULED_ARRIVAL', 'ORIG']

X_RFC = data_nine[features_RFC]
y_RFC = data_nine['DEPARTURE_DELAY_BINARY']

label_encoders = {}
for column in X_RFC.select_dtypes(include=['object']).columns:
    label_encoders[column] = LabelEncoder()
    X_RFC.loc[:, column] = label_encoders[column].fit_transform(X_RFC[column])

# Train-Test Split of dataset
X_train_RFC, X_test_RFC, y_train_RFC, y_test_RFC = train_test_split(X_RFC, y_RFC, test_size=0.2, random_state=42)

rf_classifier = RandomForestClassifier(criterion='gini', n_estimators=150, random_state=32)
rf_classifier.fit(X_train_RFC, y_train_RFC)

y_pred_RFC = rf_classifier.predict(X_test_RFC)

accuracy_RFC = accuracy_score(y_test_RFC, y_pred_RFC)
print(f"Accuracy: {accuracy_RFC:.2f}")

print("Confusion Matrix:")
print(confusion_matrix(y_test_RFC, y_pred_RFC))

print("Classification Report:")
print(classification_report(y_test_RFC, y_pred_RFC))
```

```
→ Accuracy: 0.77
Confusion Matrix:
[[ 6164 13399]
 [ 5468 55908]]
Classification Report:
              precision    recall  f1-score   support

     0       0.53         0.32         0.40       19563
     1       0.81         0.91         0.86       61376

 accuracy          0.77         0.77         0.77       80939
 macro avg         0.67         0.61         0.63       80939
 weighted avg         0.74         0.77         0.74       80939
```

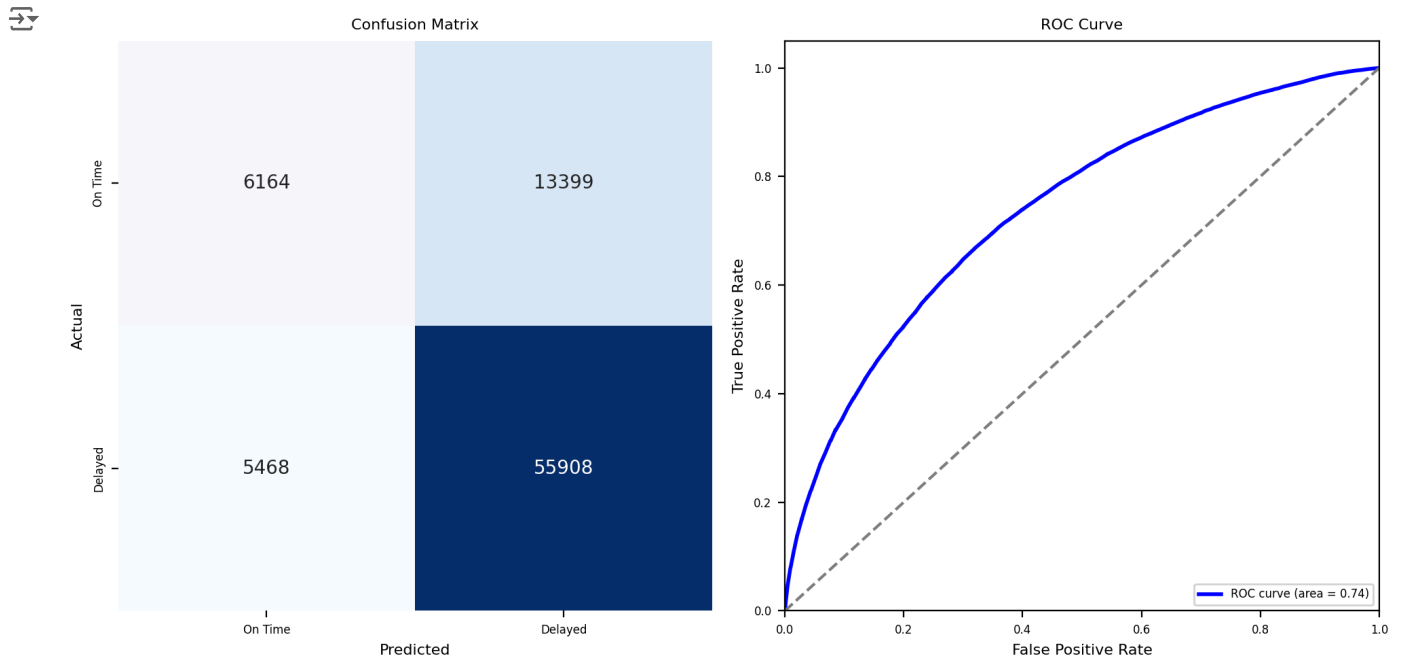
```
# Chart Data
conf_matrix = confusion_matrix(y_test_RFC, y_pred_RFC)
y_prob_RFC = rf_classifier.predict_proba(X_test_RFC)[:, 1]
fpr, tpr, _ = roc_curve(y_test_RFC, y_prob_RFC)
roc_auc = auc(fpr, tpr)
```

```
# Confusion Matrix
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5), dpi=200)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['On Time', 'Delayed'], yticklabels=['On Time', 'Delayed'], ax=ax1)
ax1.set_title("Confusion Matrix", fontsize=8)
ax1.set_xlabel("Predicted", fontsize=8)
ax1.set_ylabel("Actual", fontsize=8)
ax1.tick_params(labelsize=6)
```

```
# ROC Curve
```

```
ax2.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
ax2.plot([0, 1], [0, 1], color='gray', linestyle='--')
ax2.set_xlim([0.0, 1.0])
ax2.set_ylim([0.0, 1.05])
ax2.set_xlabel('False Positive Rate', fontsize=8)
ax2.set_ylabel('True Positive Rate', fontsize=8)
ax2.set_title('ROC Curve', fontsize=8)
ax2.legend(loc='lower right', fontsize=6)
ax2.tick_params(labelsize=6)
plt.tight_layout()
plt.show()
```



4.2 - XGBoostRegressor - Departure Delay

```
# Filter the data
data_thirteen = data_nine[data_nine['DEPARTURE_DELAY'] >= 1]
target_XGB = 'DEPARTURE_DELAY'
X_XGB = data_thirteen.drop(['DEPARTURE_DELAY', 'AIR_SYSTEM_DELAY', 'CHK_IMM_DELAY', 'SECURITY_DELAY',
                             'AIRLINE_DELAY', 'LATE_AIRCRAFT_DELAY', 'WEATHER_DELAY'], axis=1)
y_XGB = data_thirteen[target_XGB]
X_XGB = pd.get_dummies(X_XGB, drop_first=True)

# Split the data into training and testing sets
X_train_XGB, X_test_XGB, y_train_XGB, y_test_XGB = train_test_split(X_XGB, y_XGB, test_size=0.2, random_state=42)
train_dmatrix = xgb.DMatrix(X_train_XGB, label=y_train_XGB)
test_dmatrix = xgb.DMatrix(X_test_XGB, label=y_test_XGB)

params = {
    'objective': 'reg:squarederror',
    'eval_metric': 'rmse',
    'learning_rate': 0.01,
    'max_depth': 10,
    'min_child_weight': 1,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'alpha': 0.01,
    'lambda': 0.01,
    'gamma': 0.1
}

num_boost_round = 2000
early_stopping_rounds = 50
evals = [(train_dmatrix, 'train'), (test_dmatrix, 'valid')]

evals_result = {}
model = xgb.train(params=params,
                  dtrain=train_dmatrix,
                  num_boost_round=num_boost_round,
                  evals=evals,
                  early_stopping_rounds=early_stopping_rounds,
                  evals_result=evals_result,
                  verbose_eval=True)

y_pred_train_XGB = model.predict(train_dmatrix, iteration_range=(0, model.best_iteration))
y_pred_test_XGB = model.predict(test_dmatrix, iteration_range=(0, model.best_iteration))

train_mse = mean_squared_error(y_train_XGB, y_pred_train_XGB)
test_mse = mean_squared_error(y_test_XGB, y_pred_test_XGB)

train_r2 = r2_score(y_train_XGB, y_pred_train_XGB)
test_r2 = r2_score(y_test_XGB, y_pred_test_XGB)
```

```
print(f"Train MSE: {train_mse:.2f}")
print(f"Test MSE: {test_mse:.2f}")
print(f"Train R-squared: {train_r2:.2f}")
print(f"Test R-squared: {test_r2:.2f}")
```

```

[1946] train-rmse:6.96918      valid-rmse:7.72050
[1947] train-rmse:6.96906      valid-rmse:7.72049
[1948] train-rmse:6.96861      valid-rmse:7.72035
[1949] train-rmse:6.96813      valid-rmse:7.72020
[1950] train-rmse:6.96807      valid-rmse:7.72021
[1951] train-rmse:6.96793      valid-rmse:7.72018
[1952] train-rmse:6.96782      valid-rmse:7.72017
[1953] train-rmse:6.96730      valid-rmse:7.72006
[1954] train-rmse:6.96680      valid-rmse:7.71993
[1955] train-rmse:6.96648      valid-rmse:7.71984
[1956] train-rmse:6.96624      valid-rmse:7.71977
[1957] train-rmse:6.96575      valid-rmse:7.71966
[1958] train-rmse:6.96563      valid-rmse:7.71963
[1959] train-rmse:6.96531      valid-rmse:7.71963
[1960] train-rmse:6.96518      valid-rmse:7.71958
[1961] train-rmse:6.96500      valid-rmse:7.71955
[1962] train-rmse:6.96477      valid-rmse:7.71954
[1963] train-rmse:6.96437      valid-rmse:7.71947
[1964] train-rmse:6.96423      valid-rmse:7.71946
[1965] train-rmse:6.96396      valid-rmse:7.71939
[1966] train-rmse:6.96385      valid-rmse:7.71937
[1967] train-rmse:6.96376      valid-rmse:7.71935
[1968] train-rmse:6.96363      valid-rmse:7.71933
[1969] train-rmse:6.96334      valid-rmse:7.71926
[1970] train-rmse:6.96275      valid-rmse:7.71913
[1971] train-rmse:6.96254      valid-rmse:7.71908
[1972] train-rmse:6.96228      valid-rmse:7.71903
[1973] train-rmse:6.96209      valid-rmse:7.71899
[1974] train-rmse:6.96162      valid-rmse:7.71891
[1975] train-rmse:6.96143      valid-rmse:7.71894
[1976] train-rmse:6.96120      valid-rmse:7.71889
[1977] train-rmse:6.96100      valid-rmse:7.71882
[1978] train-rmse:6.96064      valid-rmse:7.71867
[1979] train-rmse:6.96049      valid-rmse:7.71867
[1980] train-rmse:6.96020      valid-rmse:7.71862
[1981] train-rmse:6.96012      valid-rmse:7.71861
[1982] train-rmse:6.95986      valid-rmse:7.71858
[1983] train-rmse:6.95969      valid-rmse:7.71858
[1984] train-rmse:6.95948      valid-rmse:7.71855
[1985] train-rmse:6.95922      valid-rmse:7.71855
[1986] train-rmse:6.95903      valid-rmse:7.71848
[1987] train-rmse:6.95881      valid-rmse:7.71848
[1988] train-rmse:6.95857      valid-rmse:7.71844
[1989] train-rmse:6.95820      valid-rmse:7.71843
[1990] train-rmse:6.95796      valid-rmse:7.71840
[1991] train-rmse:6.95781      valid-rmse:7.71838
[1992] train-rmse:6.95772      valid-rmse:7.71837
[1993] train-rmse:6.95713      valid-rmse:7.71831
[1994] train-rmse:6.95708      valid-rmse:7.71829
[1995] train-rmse:6.95700      valid-rmse:7.71828
[1996] train-rmse:6.95686      valid-rmse:7.71824
[1997] train-rmse:6.95678      valid-rmse:7.71825
[1998] train-rmse:6.95613      valid-rmse:7.71812
[1999] train-rmse:6.95597      valid-rmse:7.71808
Train MSE: 48.39
Test MSE: 59.57
Train R-squared: 0.60
Test R-squared: 0.50

```

```
# Actual vs Predicted Scatter Plot
```

```

plt.rcParams['figure.dpi'] = 150
plt.rcParams['figure.figsize'] = [12, 5]
plt.rcParams['font.size'] = 10
fig, axes = plt.subplots(1, 2)
axes[0].scatter(y_test_XGB, y_pred_test_XGB, alpha=0.5, color='blue')
axes[0].plot([y_test_XGB.min(), y_test_XGB.max()], [y_test_XGB.min(), y_test_XGB.max()], 'r--', lw=2)
axes[0].set_xlabel('Actual DEPARTURE_DELAY')
axes[0].set_ylabel('Predicted DEPARTURE_DELAY')
axes[0].set_title('Actual vs Predicted (Test Set)')
axes[0].grid(True)

```

```
# Learning Curve (RMSE vs Boosting Rounds)
```

```

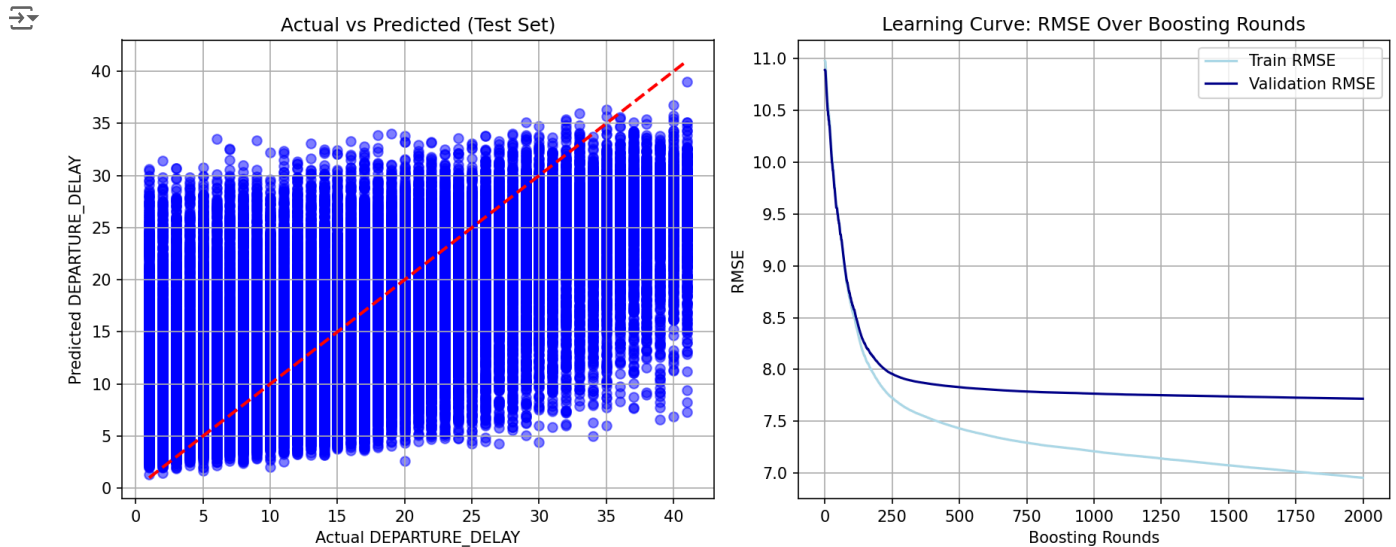
train_rmse = evals_result['train']['rmse']
valid_rmse = evals_result['valid']['rmse']

```

```

axes[1].plot(range(num_boost_round), train_rmse, label='Train RMSE', color='lightblue')
axes[1].plot(range(num_boost_round), valid_rmse, label='Validation RMSE', color='darkblue')
axes[1].set_xlabel('Boosting Rounds')
axes[1].set_ylabel('RMSE')
axes[1].set_title('Learning Curve: RMSE Over Boosting Rounds')
axes[1].legend()
axes[1].grid(True)
plt.tight_layout()
plt.show()

```



✓ 5 - PRODUCTION USE CASE FOR PREDICTIONS

5.1 - RandomForestClassifier

```
# Implementation of trained RF classifier model
```

```

def get_user_input():
    try:
        print("Please enter the following flight details:")
        day = int(input("Day of the month (1-31): "))
        day_of_week = int(input("Day of the week (1 = Monday, 7 = Sunday): "))
        flight_number = input("Flight number (e.g., '1234'): ")
        iata_code = input("Airline IATA code (e.g., 'AA' for American Airlines): ").upper()
        scheduled_departure = int(input("Scheduled departure time (in HHMM format, e.g., 1400 for 2:00 PM): "))
        scheduled_arrival = int(input("Scheduled arrival time (in HHMM format, e.g., 1600 for 4:00 PM): "))
        origin_airport = input("Origin airport IATA code (e.g., 'JFK'): ").upper()
        destination_airport = input("Destination airport IATA code (e.g., 'LAX'): ").upper()

        if not (1 <= day <= 31):
            raise ValueError("Day must be between 1 and 31.")
        if not (1 <= day_of_week <= 7):
            raise ValueError("Day of the week must be between 1 and 7.")
        if not (0 <= scheduled_departure <= 2359):
            raise ValueError("Scheduled departure time must be between 0000 and 2359.")

        user_input = {
            'DAY': day,
            'DAY_OF_WEEK': day_of_week,
            'FLIGHT_NUMBER': flight_number,
            'IATA_CODE': iata_code,
            'SCHEDULED_DEPARTURE': scheduled_departure,
            'SCHEDULED_ARRIVAL': scheduled_arrival,
            'ORIGIN_AIRPORT': origin_airport,
            'DESTINATION_AIRPORT': destination_airport,
        }

        return user_input
    except ValueError as e:
        print(f"Invalid input: {e}")
        return get_user_input()

def preprocess_and_predict(user_input, model, label_encoders):

```

```

user_input_df = pd.DataFrame([user_input])
for column in user_input_df.select_dtypes(include=['object']).columns:
    if column in label_encoders:
        try:
            user_input_df[column] = label_encoders[column].transform(user_input_df[column])
        except ValueError:
            print(f"Error: The input '{column}' contains a value not seen during training: {user_input_df[column]}.
            return "Unable to make prediction due to unseen category in input."

prediction = model.predict(user_input_df)
if prediction[0] == 1:
    return "The flight is predicted to be delayed."
else:
    return "The flight is predicted to be on time."

user_input = get_user_input()
try:
    result = preprocess_and_predict(user_input, rf_classifier, label_encoders)
    print(result)
except Exception as e:
    print(f"Error during prediction: {e}")

```

➡ Please enter the following flight details:

Day of the month (1-31): 23

Day of the week (1 = Monday, 7 = Sunday): 6

Flight number (e.g., '1234'): 1167

Airline IATA code (e.g., 'AA' for American Airlines): DL

Scheduled departure time (in HHMM format, e.g., 1400 for 2:00 PM): 1230

Scheduled arrival time (in HHMM format, e.g., 1600 for 4:00 PM): 1840

Origin airport IATA code (e.g., 'JFK'): PHL

Destination airport IATA code (e.g., 'LAX'): DFW

The flight is predicted to be delayed.

5.2 - XGBoostRegressor

Implementation of trained XGBoostRegressor model

```

def get_user_input():
    print("Please enter the following flight details:")
    try:
        day = int(input("Day of the month (1-31): "))
        if not (1 <= day <= 31):
            raise ValueError("Day must be between 1 and 31.")
        day_of_week = int(input("Day of the week (1 = Monday, 7 = Sunday): "))
        if not (1 <= day_of_week <= 7):
            raise ValueError("Day of the week must be between 1 and 7.")
        flight_number = input("Flight number (e.g., '1234'): ")
        iata_code = input("Airline IATA code (e.g., 'AA' for American Airlines): ").upper()
        scheduled_departure = int(input("Scheduled departure time (in HHMM format, e.g., 1400 for 2:00 PM): "))
        if not (0 <= scheduled_departure <= 2359):
            raise ValueError("Scheduled departure time must be between 0000 and 2359.")
        scheduled_arrival = int(input("Scheduled arrival time (in HHMM format, e.g., 1600 for 4:00 PM): "))
        if not (0 <= scheduled_arrival <= 2359):
            raise ValueError("Scheduled arrival time must be between 0000 and 2359.")
        origin_airport = input("Origin airport IATA code (e.g., 'JFK'): ").upper()
        destination_airport = input("Destination airport IATA code (e.g., 'LAX'): ").upper()

        user_input = {
            'DAY': day,
            'DAY_OF_WEEK': day_of_week,
            'FLIGHT_NUMBER': flight_number,
            'IATA_CODE': iata_code,
            'SCHEDULED_DEPARTURE': scheduled_departure,
            'SCHEDULED_ARRIVAL': scheduled_arrival,
            'ORIGIN_AIRPORT': origin_airport,
            'DESTINATION_AIRPORT': destination_airport
        }
        return user_input
    except ValueError as e:
        print(f"Error: {e}")
        return get_user_input()

def preprocess_user_input(user_input, X_columns):
    user_input_df = pd.DataFrame([user_input])

```